

MobileNet-v2 on CIFAR-10

CS6886 - System Engineering for Deep Learning | Assignment 2

Surya Harshith Balla | IIT Madras | 8 September 2026

Q1. Training baseline

(a) Data preparation

The CIFAR-10 split is 45,000 train, 5,000 validation and 10,000 test images. Each class has 4,500 training and 500 validation images, selected with a fixed seed. Normalization uses training pixels only: RGB mean = (0.491840, 0.482541, 0.446820), std. = (0.247051, 0.243542, 0.261692).

For training: reflect padding of 4 pixels, random 32 x 32 crop, horizontal flip ($p = 0.5$), tensor conversion, normalization and random erasing ($p = 0.25$, area 0.02-0.15, aspect ratio 0.5-2, fill 0). Validation and test images only need tensor conversion and normalization.

(b) Model and training strategy

Setting	Value
Model	torchvision MobileNet-v2, trained from scratch; width 1.0; dropout 0.2; 10 output classes
CIFAR-10 change	First convolution stride 1; remaining stage strides unchanged; 2,236,682 parameters
BatchNorm	epsilon = 1e-5; momentum = 0.1
Training	300 epochs; batch 128; SGD, momentum 0.9, Nesterov
Learning rate	0.1; 5-epoch linear warmup, then cosine decay to 1e-5
Regularization	Weight decay 5e-4, excluding BN/bias; label smoothing 0.05; gradient clipping at norm 5
Execution	Seed 42; AMP during training; FP32 evaluation; deterministic cuDNN

(c) Test accuracy, curves and failure modes

Epoch 284 had the best val. accuracy: 93.88%. That checkpoint gave 92.64% test top-1 accuracy (9,264 / 10,000), with test cross-entropy 0.2849. Training was interrupted once and resumed using the saved optimizer and random states.

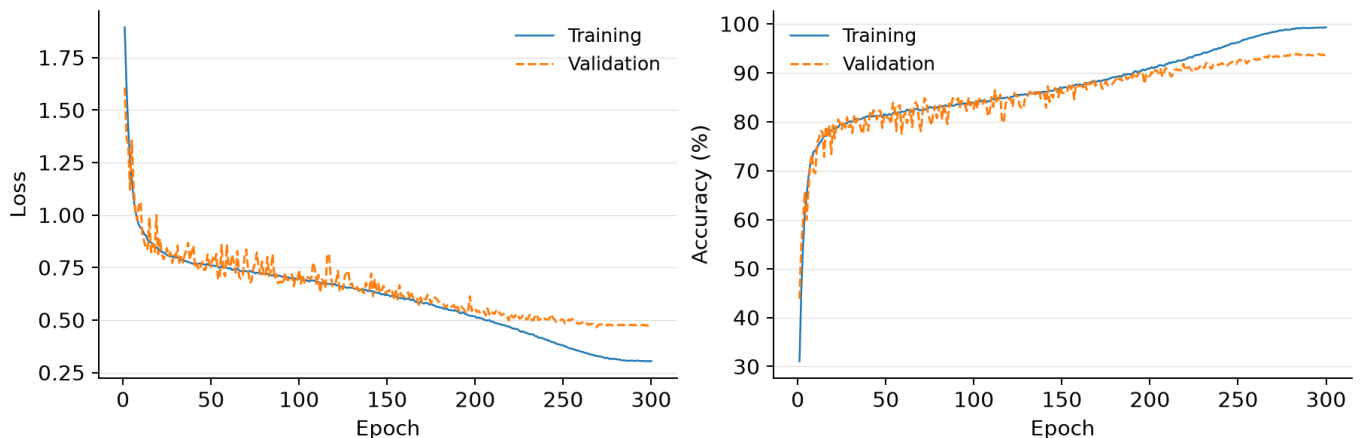


Figure 1. Loss and accuracy over 300 epochs. Both plotted losses use label smoothing 0.05. The final test loss uses no smoothing.

At the last epoch, train accuracy was 99.28% and val. accuracy was 93.82%. The gap suggests some overfitting, though training also uses augmentation. So 99% training accuracy alone doesn't tell the full story; validation and test scores matter too. This was a single-seed run, so variation across seeds wasn't checked.

Q2. Model compression implementation

(a) Configurable method and design

src/compression.py handles compression. Weight and activation bits can be set separately to 2, 4, 8 or 32. The code uses tensor operations, rounding, clipping and bit shifts. No ready-made compression or quantization API is used. Every setting uses the same trained baseline, with no fine-tuning.

Weights use symmetric quantization with one scale per output channel. A single scale for the whole layer could be too coarse for smaller channels, especially depthwise filters. For b-bit weights in output channel c:

```
L = 2**(b - 1) - 1
s[c] = max(abs(W[c])) / L
q[c] = clip(round(W[c] / s[c]), -L, L)
w_hat[c] = s[c] * q[c]
```

For an all-zero channel, the scale is 1. Adding L makes the signed codes nonnegative before packing. One byte holds two 4-bit codes or four 2-bit codes. At 2 bits, this symmetric scheme uses three signed levels, leaving one bit pattern unused.

After quantizing weights, 1,000 training images without augmentation are used to find the activation ranges at Conv/Linear inputs. Each location gets one affine scale and zero point. The range is adjusted to include zero:

```
Q = 2**b - 1
s = (high - low) / Q
z = clip(round(-low / s), 0, Q)
q = clip(round(x / s + z), 0, Q)
x_hat = (q - z) * s
```

If the range has zero width, the scale is 1. Min/max calibration is easy to implement, but a few outliers can widen the range. This leaves less precision for most values. Rounding and clipping errors can build up at very low bit-widths.

(b) Layers and exceptions

In eval mode, each adjacent Conv/BatchNorm pair is folded into one convolution. Each filter is multiplied by $\gamma / \sqrt{\text{running_variance} + \text{epsilon}}$; the bias is adjusted using the running mean and beta. BatchNorm is then replaced by an identity. Intermediate Conv/Linear weights are quantized, including pointwise and depthwise convolutions.

The first convolution (features.0.0) and classifier (classifier.1) keep their weights and inputs in FP32. All biases also stay FP32. Hooks quantize the other Conv/Linear inputs, while residual additions and other arithmetic stay in FP32. Flags can change the edge-layer exceptions or turn off BN folding. Q3 keeps these choices fixed.

(c) Storage overhead

Part of final W4A8 file	Bytes
Packed weight codes	1,094,448
FP32 per-channel weight scales	68,096
Raw FP32 edge weights and biases	122,920
Header, including activation ranges, scales and zero points	20,033
Total actual file size	1,305,497

The .mq file has a 4-byte signature, 8-byte header length, JSON header and tensor data. The table counts all of them. The activation estimate allows a 4-byte scale and 4-byte zero point at each quantized location. The model file already includes their JSON representation. Model size and activation storage are separate measurements.

Q3. Compression results

(a) Compression levels

The 3 x 3 grid tries 8, 4 and 2 bits for both weights and activations. All nine settings use the epoch-284 checkpoint. Calibration uses 1,000 training images, 100 per class (seed 42). Each setting is checked on the same 5,000 validation images, 500 per class (order seed 43). The saved calibration and validation indices don't overlap. Q3 uses no test images.

(b) Accuracy comparison

Setting	Val. acc. (%)	Drop (pp)	Size (MB)	Model ratio	Activation ratio
FP32	93.88	0.00	9.084	1.00x	1.00x
W8A8	93.88	0.00	2.400	3.78x	3.90x
W8A4	90.16	3.72	2.399	3.79x	7.57x
W8A2	10.00	83.88	2.399	3.79x	14.25x
W4A8	90.96	2.92	1.305	6.96x	3.90x
W4A4	86.86	7.02	1.305	6.96x	7.57x
W4A2	10.00	83.88	1.305	6.96x	14.25x
W2A8	10.26	83.62	0.758	11.98x	3.90x
W2A4	9.94	83.94	0.758	11.99x	7.57x
W2A2	10.04	83.84	0.758	11.99x	14.25x

Model ratio = 9,083,592 original parameter/buffer bytes divided by the complete packed file size. It includes BN folding and all file overheads. Q3 JSON/W&B calls this `weight_compression_ratio`; Q4 gives the separate weights-only ratio. Activation ratios estimate packed Conv/Linear input occurrences, not peak GPU memory.

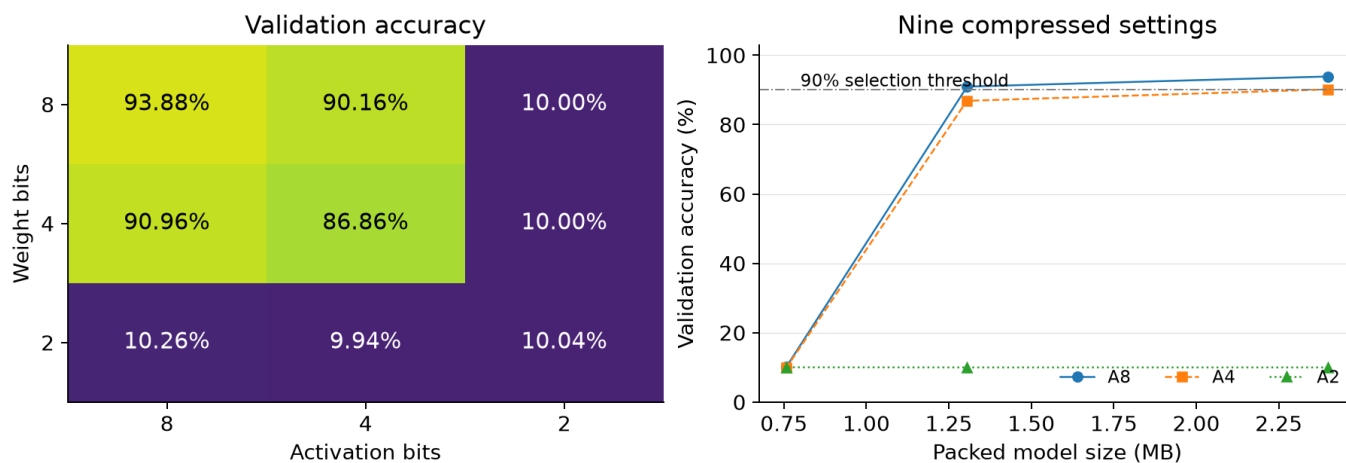


Figure 2. Val. accuracy for the nine settings, compared with packed model size.

W8A8 keeps 93.88% val. accuracy at 2.400 MB. W4A8 gives 90.96% at 1.305 MB. W8A4 gives 90.16%, with a larger saving in activation storage. W4A4 falls to 86.86%. Settings with 2-bit weights or activations are close to 10% chance accuracy. They save more space, but the accuracy loss is too large here.

Each packed file was reloaded before evaluation. Its logits matched the model before export exactly on a validation batch. This checks that saving and loading work correctly; it doesn't mean quantization preserves the FP32 model's predictions.

Q3(b). W&B Parallel Coordinates chart

W&B has ten finished runs: FP32 and the nine compressed settings. Its native Parallel Coordinates panel has six axes: weight bits, activation bits, model ratio, activation ratio, model size in MB and val. accuracy.

[Open the native W&B Parallel Coordinates chart](#)

Project: harshith7946-indian-institute-of-technology-madras/cs6886-assignment2-q3

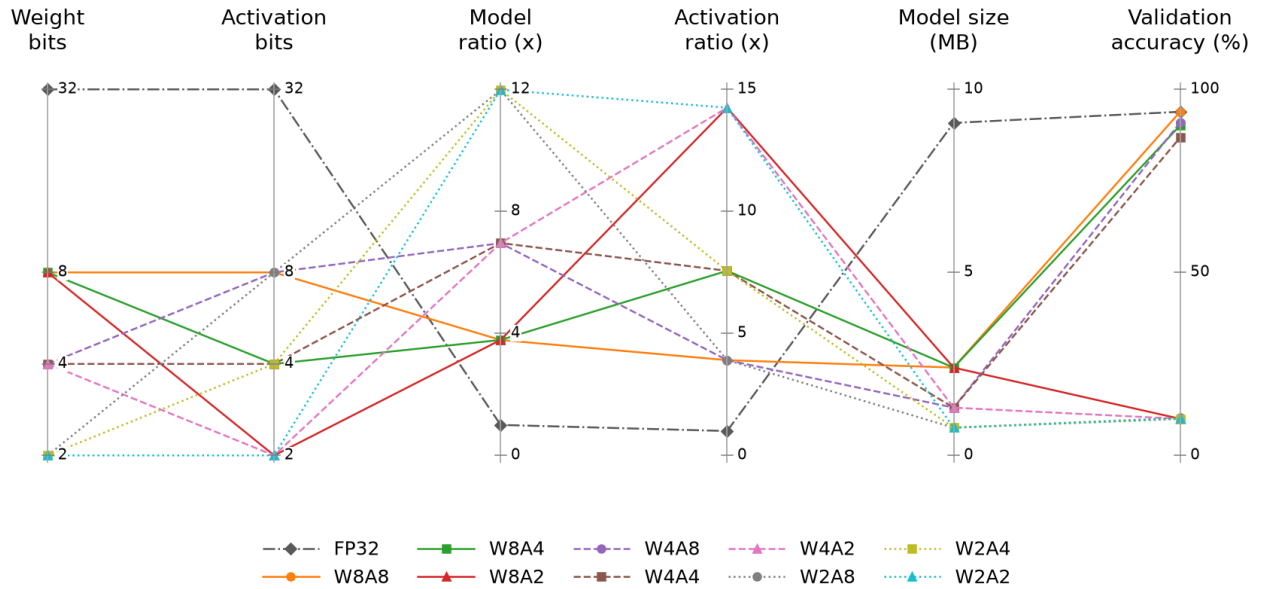


Figure 3. Local Matplotlib plot of the same ten runs. The native W&B chart is linked above.

The uploaded settings and metrics match the local results. Reloading the W&B report through its API confirmed that the native six-axis panel exists. results/wandb.json stores the report and run links.

The main trade-off is clear: the 2-bit settings save the most space but give almost chance accuracy. W4A8 is the smallest model that still reaches at least 90% val. accuracy. That is the selection rule used in Q4.

These results come from one baseline and one calibration subset. Tests across multiple seeds, integer-kernel timing and runtime-memory measurements weren't done. The saved weights are packed, but inference decodes them to FP32 and simulates activation quantization in floating point.

Q4. Compression analysis

The final choice is W4A8: the smallest model with at least 90% val. accuracy. The choice and file hash were saved before test evaluation, under selection in results/final.json. All four parts below use this same configuration.

(a) Weight compression ratio

Original Conv/Linear weights take 8,810,240 bytes. Their packed codes, scales and FP32 edge weights take 1,217,200 bytes after compression. Counting the full 20,033-byte header as weight overhead gives:

$$\begin{aligned}\text{Weight ratio} &= 8,810,240 / (1,217,200 + 20,033) \\ &= 7.12\times\end{aligned}$$

Scales and metadata are included in this ratio. BN parameters and biases aren't part of the weights-only numerator or tensor payload. They are accounted for in the complete model measurement below.

(b) Activation compression ratio and measurement

For one 32 x 32 image, the input shape is recorded at each Conv/Linear location. Adding these input occurrences gives 2,177,536 FP32 bytes. Using 8 bits at compressed locations and 32 bits at the two protected inputs gives 557,440 packed bytes. The 51 quantized locations need another 51 x 8 = 408 bytes for scales and zero points.

$$\begin{aligned}\text{Activation ratio} &= 2,177,536 / (557,440 + 408) \\ &= 3.90\times\end{aligned}$$

If several layers use an input, it is counted each time. This estimates total activation storage across the selected inputs, not peak memory or tensors live at the same time. Inference still runs in FP32, so this estimate doesn't mean the code saves that much activation memory at runtime.

(c) Quantized model accuracy

Measurement	Value
Validation top-1 accuracy used for selection	90.96%
Final test top-1 accuracy	90.13% (9,013 / 10,000)
Baseline test top-1 accuracy	92.64%
Test accuracy decrease	2.51 percentage points
Quantized test cross-entropy	0.3783

Cat has the lowest class accuracy (72.3%); automobile has the highest (95.3%). A fresh final evaluation saves predictions and a confusion matrix to check these results. The test scores weren't used to choose another configuration.

(d) Final model size

The packed file is 1,305,497 bytes = 1.305497 MB, approx. 1.31 MB (decimal MB). The original state tensors take 9,083,592 bytes, so the complete model ratio is 6.96x. This includes all overheads listed in Q2(c). It is separate from the weights-only ratio in Q4(a).

Q5. Reproducibility and repository

(a) Code organization

File or folder	Purpose
src/data.py	Prepare the split and training-only normalization
src/train.py	Build, train, save and resume the baseline
src/compression.py	Quantize, fold BN and pack model weights
src/evaluate.py	Compare settings and test the chosen model
src/report.py	Verify results and publish the W&B chart
tests/	Quantization, model reload and checkpoint tests
models/; results/; report/	Two models; measured results and plots; submission PDF

Five source files cover Q1-Q5, and one test file has eight checks. Comments explain the equations, bit packing and layer exceptions. models/ keeps the baseline and W4A8; new experiments go to runs/. The other compressed models can be regenerated when needed.

(b) Commands, environment and seeds

Tested on Windows, Python 3.14.3 and an RTX 3050 Laptop GPU (4 GB). requirements.txt pins the direct dependencies: torch 2.14.0+cu126, torchvision 0.29.0+cu126, NumPy 2.5.2, Matplotlib 3.11.1, Pillow 12.3.0, psutil 7.2.2, wandb 0.29.0 and wandb-workspaces 0.4.11.

After activating a Python 3.14 virtual environment, run these commands from the repo root:

```
python -m pip install -r requirements.txt
python -m src.data --download
python -m unittest discover -s tests -v
python -m src.report
python -m src.evaluate final --output-dir runs/final_recheck
```

The last command reloads and tests W4A8. It detects CUDA automatically; --device cpu also works. Setup and the first dataset download need internet. After that, experiments work offline. The README has the commands for training, resuming, the full sweep and publishing to W&B.

Training, splitting and calibration use seed 42; validation order uses seed 43. cuDNN is deterministic, with benchmarking off. After reorganizing the code, all nine packed models matched byte for byte, and all 10,000 test predictions matched. All eight tests passed. The split indices and normalization matched too. Different hardware or versions may still change the results.

(c) GitHub repository

<https://github.com/harshith0518/cs6886-assignment-2-mobilenetv2>

The PDF is the Moodle submission file. Repository and W&B access must be available to the evaluator if either is private.

References

[torchvision: MobileNet-v2 source](#)

[CIFAR-10 dataset](#)

[Jacob et al.: Quantization and Training of Neural Networks \(2018\)](#)

[W&B: Parallel Coordinates panels](#)